

Module 02: Agents — Implementing the Active Inference Agent

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Implement an **Active Inference agent** class that encapsulates perception, action, and belief updating.
2. Code the **belief update loop** — observe, infer, act, repeat.
3. Run the agent in a simple **environment** and log its behavior.

Introduction

Module 01 set up the data structures. Now we build the agent itself — a class that interacts with an environment through the perception-action loop.

Key Concepts

1. The Agent Class

```

import numpy as np

class ActiveInferenceAgent:
    """A discrete-state Active Inference agent."""

    def __init__(self, A, B, C, D, policy_len=1, gamma=1.0):
        self.A = A
        self.B = B
        self.C = C
        self.D = D.copy()
        self.gamma = gamma # policy precision

        self.num_states = A.shape[1]
        self.num_obs = A.shape[0]
        self.num_actions = B.shape[2]
        self.policy_len = policy_len

        # Current beliefs
        self.qs = D.copy() # posterior over states

        # Generate all possible policies
        self.policies = self._generate_policies()

        # Logging
        self.belief_history = []
        self.action_history = []
        self.obs_history = []

    def _generate_policies(self):
        """Generate all possible single-step policies."""
        return np.arange(self.num_actions).reshape(-1, 1)

```

```

def infer_states(self, observation):
    """Update beliefs given a new observation (Bayesian update)."""
    likelihood = self.A[observation, :]
    self.qs = likelihood * self.qs
    self.qs = self.qs / self.qs.sum()

    # Log
    self.obs_history.append(observation)
    self.belief_history.append(self.qs.copy())
    return self.qs

def infer_policies(self):
    """Evaluate policies using Expected Free Energy."""
    G = np.zeros(len(self.policies))
    for i, policy in enumerate(self.policies):
        G[i] = self._compute_efe(policy)

    # Softmax policy selection
    pi = np.exp(-self.gamma * G)
    pi = pi / pi.sum()
    return pi, G

def _compute_efe(self, policy):
    """Compute Expected Free Energy for a policy."""
    G = 0
    qs_pred = self.qs.copy()

    for action in policy:
        # Predict next state
        qs_pred = self.B[:, :, action] @ qs_pred
        # Predict observation

```

```

qo_pred = self.A @ qs_pred

# Pragmatic value:  $-E[\log P(o)]$ 
pragmatic = -(qo_pred * self.C).sum()

# Epistemic value: mutual information
epistemic = 0
for s in range(self.num_states):
    if qs_pred[s] > 1e-16:
        po_given_s = self.A[:, s]
        epistemic += qs_pred[s] * (
            po_given_s * (np.log(po_given_s + 1e-16)
                - np.log(qo_pred + 1e-16))
        ).sum()

G += pragmatic - epistemic

return G

def select_action(self):
    """Select an action based on policy evaluation."""
    pi, G = self.infer_policies()
    action = np.random.choice(len(self.policies), p=pi)
    self.action_history.append(action)
    return int(self.policies[action][0])

```

2. The Environment

```
class SimpleEnvironment:
    """A simple environment the agent interacts with."""

    def __init__(self, true_A, true_B, initial_state=0):
        self.A = true_A # true observation model
        self.B = true_B # true transition model
        self.state = initial_state

        self.state_history = [initial_state]

    def step(self, action):
        """Apply action, transition state, generate observation."""
        # Transition
        transition_probs = self.B[:, self.state, action]
        self.state = np.random.choice(len(transition_probs), p=transition_probs)

        # Generate observation
        obs_probs = self.A[:, self.state]
        observation = np.random.choice(len(obs_probs), p=obs_probs)

        self.state_history.append(self.state)
        return observation

    def reset(self, state=0):
        """Reset environment to initial state."""
        self.state = state
        self.state_history = [state]
```

3. The Simulation Loop

```
def run_simulation(agent, env, num_steps=20):  
    """Run the agent-environment loop."""  
    # Initial observation  
    obs = np.random.choice(agent.num_obs, p=env.A[:, env.state])  
  
    for t in range(num_steps):  
        # 1. Infer states from observation  
        agent.infer_states(obs)  
  
        # 2. Select action  
        action = agent.select_action()  
  
        # 3. Execute action in environment  
        obs = env.step(action)  
  
        print(f"t={t}: obs={obs}, action={action}, "  
              f"beliefs={agent.qs.round(2)}, "  
              f"true_state={env.state}")  
  
    return agent, env
```

Summary

The Active Inference agent class encapsulates belief updating, policy evaluation via EFE, and action selection. The environment generates observations and transitions states. The simulation loop connects them through the perception-action cycle — observe, infer states, select action, act, repeat.

Further Reading

- Heins, C. et al. (2022). pymdp: A Python library for active inference in discrete state spaces. *JOSS*, 7(73), 4098.
- Smith, R. et al. (2022). A step-by-step tutorial on active inference. *Journal of Mathematical Psychology*, 107, 102632.